FinPilot AI: A Personal Finance Agent with Deterministic Tools and Explicit Approval

Team member 1
Institution: _____
Email: _____

Team member 2
Institution: _____
Email: _____

***Abstract*—Personal finance assistants need to turn questions into useful actions while retaining numerical accuracy and user control. This paper presents FinPilot AI, a working synthetic-account sandbox for budgeting, transaction analysis, recurring bills, and explainable spending insights. The system separates optional language-model tool planning from deterministic financial calculations and authorization. Read tools return ledger-derived evidence; proposal tools prepare exact changes; mutation tools require an expiring, single-use token bound to the user and canonical payload. Budget changes, approval consumption, and audit records commit atomically. A keyless deterministic agent completes the same demonstration without an external model. On a fixed 90-day dataset containing 134 synthetic transactions, the principal scenario explains a dining-spending increase and proposes a next-month budget of USD 518.48 from USD 648.10 of recorded spending. Verification comprises 61 TypeScript domain/API tests, 4 Chromium scenarios, and 8 FastAPI tests. These checks establish bounded functional and security invariants; they do not establish financial-advice quality, production scalability, or regulatory compliance. Source, execution evidence, and reproducible setup accompany the implementation.**

***Index Terms*—personal finance, financial agents, deterministic tools, explicit approval, explainability, sandbox evaluation**

I. Introduction

An assistant that describes spending without completing an approved financial task provides limited operational value. Conversely, granting a language model unrestricted write access creates a permission boundary that is difficult for a user to inspect. Personal finance therefore requires a workflow that connects explanation, review, execution, and evidence.

FinPilot AI addresses Topic A, Personal Finance Assistant, in the 2026 Shenzhen International FinTech Competition. The supplied appendix asks for actionable budgeting, bill analysis, and spending insights, and weights task completion at 40%, security and compliance at 30%, and innovation and interaction at 30% [1]. Our implementation operates exclusively on synthetic sandbox records. It performs no bank connection, payment, securities transaction, or external notification delivery.

The engineering contributions are: (1) a working tool-based finance workflow with a deterministic fallback; (2) an explicit authorization protocol that binds approval to the exact reviewed payload; and (3) a reproducible evaluation package linking calculations, browser actions, and sanitized execution records. These contributions describe an implemented system; we do not claim a new financial forecasting algorithm or a benchmark advantage over existing assistants.

II. Related Work and Design Position

ReAct studies language-model reasoning interleaved with environment actions [2]. FinPilot adopts the practical idea that an assistant should invoke inspectable tools, but its optional provider uses a bounded planning request rather than an unrestricted iterative reasoning loop. The planner selects operations; application code supplies numeric results and the final grounded response.

OWASP identifies excessive permissions and autonomy as sources of harmful agent actions [3]. The design responds by exposing only narrow read/proposal functions to the model and placing the write decision behind explicit user review. NIST AI RMF offers broader risk-management guidance [4]; it informs the separation of implemented controls, measured behavior, and residual risks rather than serving as a certification claim. The provider integration uses the Responses function-calling interface [5]. The paper follows IEEE conference layout conventions [6]; it is competition documentation, not an IEEE publication or endorsement.

III. Requirements and System Scope

The intended user can inspect synthetic income and expenses, upload a validated CSV, identify recurring payments, create category budgets, and ask the agent to propose a change. The principal task is: explain why dining spending increased this month and set the next month's dining budget 20% below recorded current-month dining spending. A successful run must present evidence and an exact proposal, apply only after approval, and leave finances unchanged after rejection.

Three invariants guide the implementation. First, financial amounts are signed integer minor units, with no floating-point values stored as money. Second, every protected read/write derives ownership from a verified session rather than a user-selected identifier. Third, a model response cannot authorize a financial mutation. Direct UI forms require explicit confirmation; agent-originated mutations additionally require the proposal token.

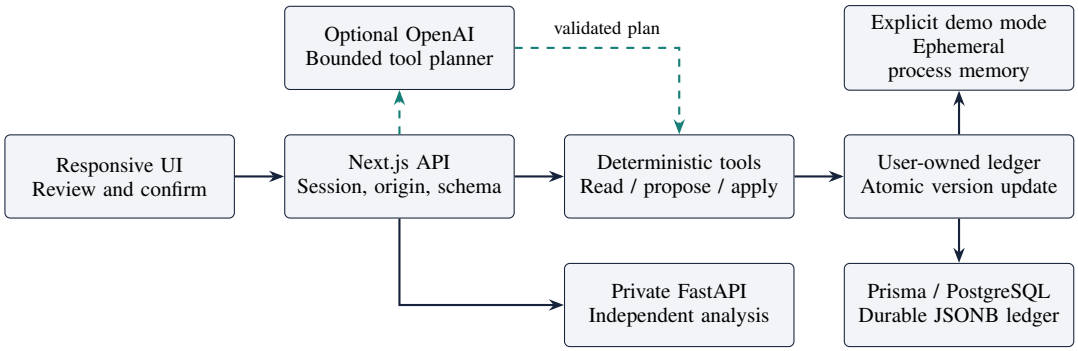


Fig. 1. Implemented architecture. Solid arrows show application/service boundaries; dashed arrows denote optional provider planning. Repository selection is explicit: PostgreSQL persistence or a visibly ephemeral single-process demo. No component connects to a real bank or payment rail.

IV. System Architecture

A. Application and Analysis Boundaries

The interface uses Next.js App Router, strict TypeScript, Tailwind CSS, a shared responsive design system, and Recharts. Routes cover the landing page, authentication, dashboard, transactions, budgets, bills, insights, agent, audit activity, and settings. The same-origin API validates requests with Zod. Merchant names and uploaded text remain untrusted data and are rendered as escaped text.

Deterministic TypeScript functions implement totals, comparisons, recurrence, anomalies, and budget calculations. A private, stateless Python FastAPI service independently recomputes monthly financial totals from authenticated user-scoped records. The application sends only transaction identifiers, dates, categories, and integer amounts. A shared service token and a three-second timeout protect this optional boundary; Python has no mutation or database authority.

B. Persistence and Atomicity

Prisma models relational users, revocable sessions, and a per-user ledger aggregate. The PostgreSQL ledger stores financial arrays, approvals, conversation records, and audit events in JSONB. A mutation reads the user's version, copies the aggregate, applies a synchronous operation, and executes an update conditioned on that version. A conflicting update retries up to five times. Consequently, approval consumption, budget/reminder change, and audit append share one committed update. The memory repository uses a mutex and the same copy-before-mutate semantics.

The aggregate deliberately favors reviewable atomicity over analytical scale. Rewriting a whole JSONB document is unsuitable for very large ledgers; normalized transaction tables and an independently protected audit store are future work. When no database is configured, the application visibly identifies ephemeral memory mode. Restarting that process loses data and sessions; a configured database failure never silently falls back.

C. Sessions and API Controls

Registered passwords use bcrypt with cost 12 and a 72-byte UTF-8 bound. An encrypted HttpOnly, SameSite cookie contains a random session token and user identity; the server stores only the token hash with an eight-hour expiry. Logout revokes the server record. HTTPS configuration adds the Secure cookie flag. Write requests require the expected Origin and JSON content type. Rate limits, string/amount bounds, and incremental request-body limits protect the HTTP boundary. Process-local limits require a shared gateway for multi-instance deployment.

V. Deterministic Financial Methods

A. Integer Arithmetic and Monthly Summaries

Let $a_i \in \mathbb{Z}$ be signed transaction cents for month m. Income, expenses, and net savings are

$$I_m = \sum_{i:a_i>0} a_i, \quad E_m = -\sum_{i:a_i<0} a_i, \quad S_m = I_m - E_m. \tag{1}$$

Amount sign determines cash direction; categories are labels. CSV decimals are assembled from whole/fraction digits, not multiplied from binary floating-point input. Every operand and aggregate must remain a safe integer. Calendar accounting uses UTC.

For spending $E_{m,c}$ in category c and a whole-number percentage reduction r, the proposed cent limit is

$$B_{m+1,c} = \left\lfloor \frac{E_{m,c}(100-r)+50}{100} \right\rfloor. \tag{2}$$

The numerator is evaluated with BigInt, yielding round-half-up rounding to cents. Limits must be positive and within the supported amount bound. The agent does not infer a percentage when the user has omitted one or entered an unsupported fractional reduction.

B. Comparisons and Source Evidence

The current month's spending is compared through today's UTC day with the same days of the previous month. The result contains totals, purchase counts, average purchase size, and largest purchases with transaction IDs. Historical requested months compare full recorded months. Descriptions report observed differences without inferring unobserved motives. A budget proposal uses all currently recorded spending in the source month; later purchases do not silently alter the payload already reviewed by the user.

C. Recurring Bills and Unusual Purchases

Recurring-payment inference groups expenses by normalized merchant, requires at least three observations, and checks each consecutive interval is 25–35 days. Every amount must be within 15% of the upper median. The next estimated date advances by calendar month, clamping month-end dates. The output includes supporting transaction IDs; it is an inferred pattern, not a contractual schedule.

An unusual-purchase flag requires at least five strictly earlier category expenses. With prior upper median M_c, expense x is flagged when

$$x > \max(3M_c, 10000) \tag{3}$$

in cents. Same-day purchases are excluded from the baseline. These transparent rules produce descriptive flags rather than fraud determinations. No real-world detection precision or recall is claimed.

D. Atomic CSV Import

Imports accept at most 512 KB and 2,000 rows with ordered date, merchant, amount, and category headers. Calendar dates, merchant bounds, category membership, decimal precision, and amounts are validated for the entire batch before a write. A fingerprint of date, normalized merchant, and signed amount identifies duplicates both within a batch and against existing records. Validation reruns inside the atomic mutation. This deliberately treats identical same-day purchases as duplicates, a documented limitation. Exported formula-prefixed merchant cells receive an apostrophe before CSV quoting.

VI. Tool Orchestration and Approval

A. Permission Tiers

The read tier comprises spending summaries, transaction listing, month comparison, budget status, recurring bills, and anomaly explanations. The proposal tier prepares budget changes and named bill reminders. The mutation tier comprises `apply_budget_change` and `apply_bill_reminder`; these operations are not exposed to the model. A keyless parser supports the demonstrated finance intents using the same real tool executor. Optional provider planning uses strict function schemas, an eight-call cap, a 12-second provider timeout, and a 20-second execution guard. Provider failures or malformed plans choose the fallback; valid but unsupported requests return a bounded error or clarification rather than inventing an action.

B. Exact-Payload Approval Protocol

A proposal contains user identity u, canonical payload hash h_p, token hash h_t, expiry e, and status. The client receives a cryptographically random 256-bit token once. Application state stores only its hash. Approval or rejection is accepted only when

$$u_{session} = u_{proposal}, \tag{4}$$

$$H(\mathrm{canon}(p_{request})) = h_p, \tag{5}$$

$$H(t_{request}) = h_t, \tag{6}$$

$$\mathrm{status} = \mathrm{pending}, \tag{7}$$

$$\mathrm{now} < e. \tag{8}$$

Canonical JSON sorts object keys. Token hashes are compared with a timing-safe function. Expiry is five minutes. A successful approval changes the budget/reminder, consumes the token, updates the visible timeline, and appends an audit event. Rejection consumes the token and records the decision while leaving financial arrays unchanged. Replay, cross-user, modified-payload, and expired requests fail. Optimistic version checks prevent two concurrent requests from applying the same token twice.

TABLE I
Observed synthetic golden-scenario values

Measurement	Recorded value
Dining, October 1–19	USD 648.10
Dining, September 1–19	USD 241.50
Purchase count, each period	17
Mean purchase, October / September	USD 38.12 / 14.21
Largest October dining purchase	USD 248.00
Proposed November limit, 20% lower	USD 518.48

VII. Evaluation

A. Reproducible Setup

The fixed seed date is 19 October 2026 at 12:00 UTC. Ninety calendar days produce 134 synthetic transactions across income, housing, dining, groceries, transport, utilities, shopping, and subscriptions. Node.js 24.13.0 and Python 3.14.2 were used on Windows. The deployed-framework build uses Next.js 16.4.2 and Prisma 6.19.3. Evaluation code and exact logs are supplied in the repository [7]. No paid model call, human-subject study, real bank account, or production financial dataset is included.

B. Task Completion

Table I reports the actual tool results. Spending rose because the recorded purchase sizes increased; the count remained equal. The largest current-period transaction is linked by its generated source identifier. Equation (2) maps 64,810 cents to 51,848 cents. The browser scenario first rejects the proposal and compares budgets before/after, then runs a new proposal, approves it, checks the exact target budget, and observes both decisions in the audit log. A rejected decision is therefore demonstrated as an unchanged financial state rather than merely a conversational acknowledgement.

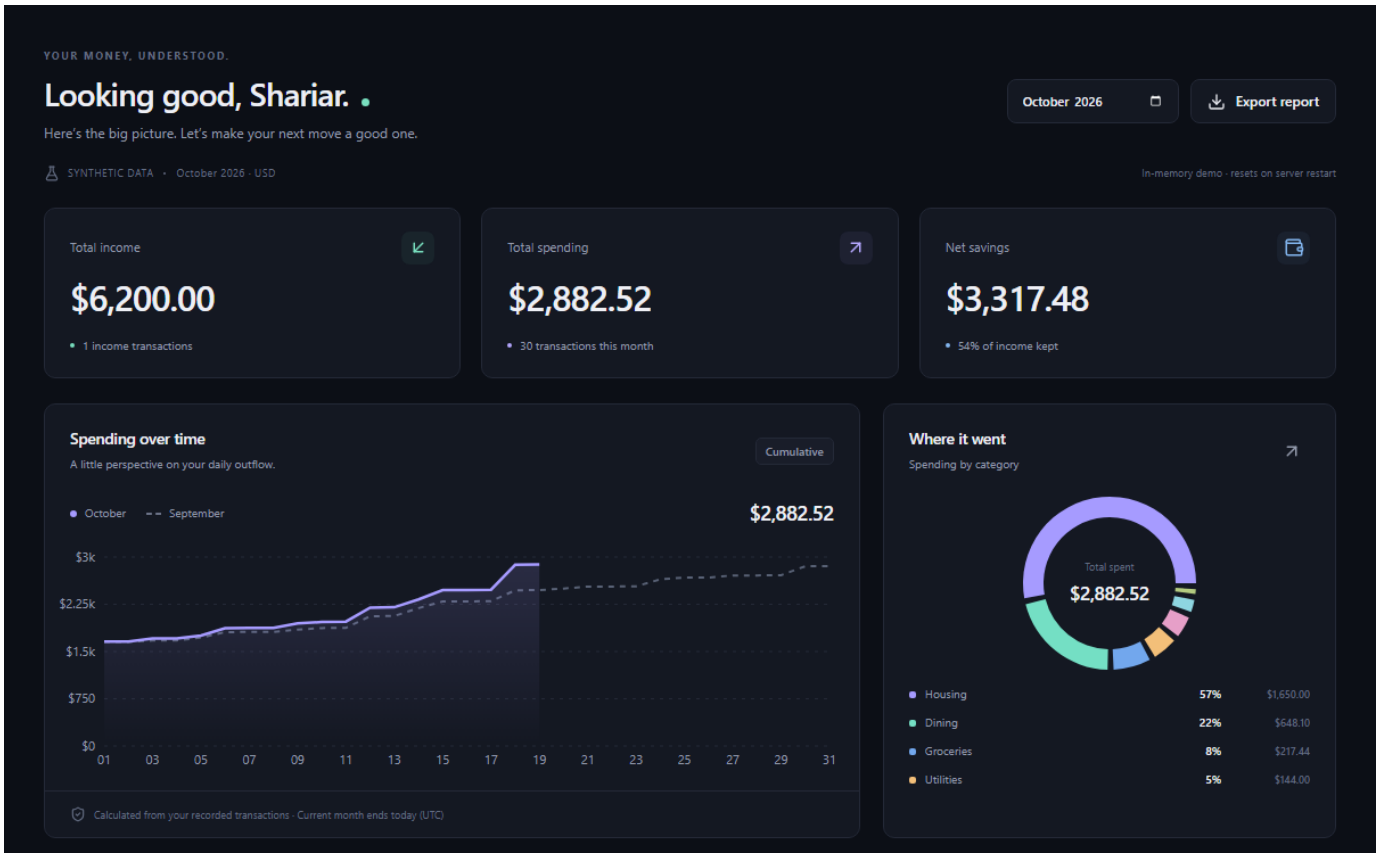


Fig. 2. Actual FinPilot dashboard captured from the running application with synthetic data. Cards and charts are computed from the ledger; screenshots are interface evidence, not generated mockups. The fixed-seed numeric experiment is reported separately in Table I.

TABLE II
EXECUTED VERIFICATION AND ITS SCOPE

Check	Observed outcome
TypeScript domain/API suite	61 passed
Chromium end-to-end scenarios	4 passed
FastAPI validation/unit suite	8 passed
Lint, strict types, seed, build	Passed; exact logs provided
Production Next.js/FastAPI link	Exact totals; missing token: 401
Live paid-provider execution	Not exercised
Local PostgreSQL connection	Not exercised locally
Remote PostgreSQL 17	Migration + 61 checks passed

C. Verification Results and Performance Scope

The tests cover integer conversion, total precision, CSV rollback/deduplication, recurrence, anomaly rules, cross-user access, session revocation, approval binding, concurrency, and browser workflows. Review regression cases additionally prevent an unknown merchant from being replaced by another bill, a missing/fractional percentage from becoming a guessed budget, and a historical summary from being described using current-month totals. Provider responses are mocked to exercise the boundary without spending or transmitting credentials.

A separate deterministic microbenchmark performs ten warm-up and 100 measured runs on newly seeded copies. Median execution is 0.534 ms and the 95th percentile is 0.827 ms on the local machine. Timing excludes seed construction, HTTP, rendering, database access, and all model latency. It is a narrow engineering measurement, not an end-to-end service-level guarantee or comparative performance claim. Machine-readable results and the exact script accompany the source.

VIII. Limitations and Deployment Conditions

Verification checks specific invariants, not arbitrary-language correctness or the absence of all vulnerabilities. The fallback supports bounded finance intents rather than general conversation. Optional model-selected proposals may still be semantically wrong; the user reviews the exact change. No model benchmark, user acceptance study, financial-advice outcome evaluation, or statistical anomaly-quality experiment was conducted.

The local environment lacked a running Docker/PostgreSQL service. Remote Ubuntu CI subsequently applied the migration to PostgreSQL 17 and passed the same 61 domain/API cases, including repository concurrency and rollback; the application job also passed build, Python, and browser checks (verified CI run 39220992552). Repeated adapter runs are not additional

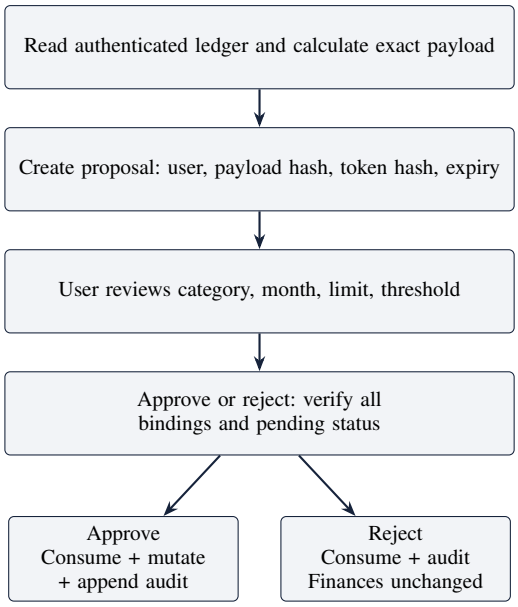


Fig. 3. Approval and rejection share an exact-payload verification step. A failed binding check cannot reach either decision commit; successful decisions consume the token exactly once.

unique tests. This does not verify target deployment or restart recovery. The JSONB aggregate, process-local rate limits, inline-script CSP allowance, session cleanup, and absence of MFA/recovery require further work before use with real customers. A trusted TLS gateway and persistent secrets are deployment requirements. No certification, regulated financial advice, or payment capability is asserted.

IX. Conclusion

FinPilot demonstrates a complete sandbox task: an expense question leads to source-backed analysis, a deterministic proposal, an explicit decision, and an auditable state transition. The principal result is the implemented separation of language planning, financial computation, and mutation authority. The supplied code, tests, screenshots, and logs make that result inspectable. Further evaluation should prioritize durable deployment verification, broader intent coverage, independent security assessment, and user studies before any real-finance use.

Reproducibility and Authorship

Source and setup: https://github.com/Shariar-Joy/finpilot-ai. All evaluation records are synthetic. Team identity, affiliations, and contributions must be completed by the two members. Implementation and documentation were prepared with AI-assisted engineering; the team must review all claims, dependencies, and the organizer's applicable independence/IP rules before making any authorship declaration. No signatures, affiliations, funding, or external endorsements are supplied by this document.

References

[1] 2026 Shenzhen International FinTech Competition Organizers, "Appendix 2: Problem description and competition rules for international track," 2026, organizer-supplied PDF, pp. 2–4. Supplied with the project brief; original document is not redistributed.

[2] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *International Conference on Learning Representations*, 2023, arXiv:2210.03629. [Online]. Available: https://arxiv.org/abs/2210.03629

[3] OWASP GenAI Security Project, "LLM06:2025 excessive agency," https://genai.owasp.org/llmrisk/llm062025-excessive-agency/, 2025, accessed: Oct. 19, 2026.

[4] National Institute of Standards and Technology, "Artificial intelligence risk management framework (AI RMF 1.0)," U.S. Department of Commerce, Tech. Rep. NIST AI 100-1, 2023. [Online]. Available: https://nvlpubs.nist.gov/nistpubs/ai/NIST.AI.100-1.pdf

[5] OpenAI, "Function calling," https://developers.openai.com/api/docs/guides/function-calling, accessed: Oct. 19, 2026.

[6] IEEE, "Authoring tools and templates," https://conferences.ieeeauthorcenter.ieee.org/write-your-paper/authoring-tools-and-templates/, accessed: Oct. 19, 2026.

[7] FinPilot AI Contributors, "FinPilot AI: Source code and reproducibility documentation," https://github.com/Shariar-Joy/finpilot-ai, 2026, see docs/execution-evidence.md and docs/evidence/verification-summary.json for verification scope.